

Mermaid - Sistema – Biblioteca

Estrutura do Sistema:

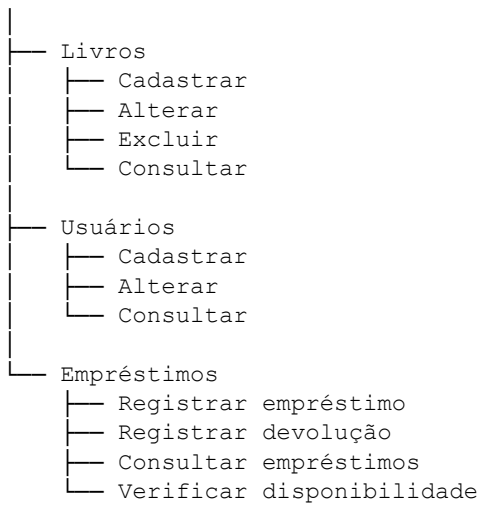


Diagrama de Caso de Uso

Atores principais:

- Bibliotecário: realiza cadastros e gerencia empréstimos.
- Usuário: consulta livros e solicita empréstimos/devoluções.

flowchart LR

```
Bibliotecario[" Bibliotecário"]
Usuario[" Usuário"]

subgraph Sistema["Sistema de Biblioteca"]
    UC1(["Cadastrar livro"])
    UC2(["Alterar livro"])
    UC3(["Excluir livro"])
    UC4(["Consultar livros"])

    UC5(["Cadastrar usuário"])
    UC6(["Alterar usuário"])
    UC7(["Consultar usuário"])

    UC8(["Registrar empréstimo"])
    UC9(["Registrar devolução"])
    UC10(["Consultar empréstimos"])
    UC11(["Verificar disponibilidade"])
end

Bibliotecario --> UC1
Bibliotecario --> UC2
Bibliotecario --> UC3
Bibliotecario --> UC4

Bibliotecario --> UC5
Bibliotecario --> UC6
Bibliotecario --> UC7
```

```
Bibliotecario --> UC8  
Bibliotecario --> UC9  
Bibliotecario --> UC10
```

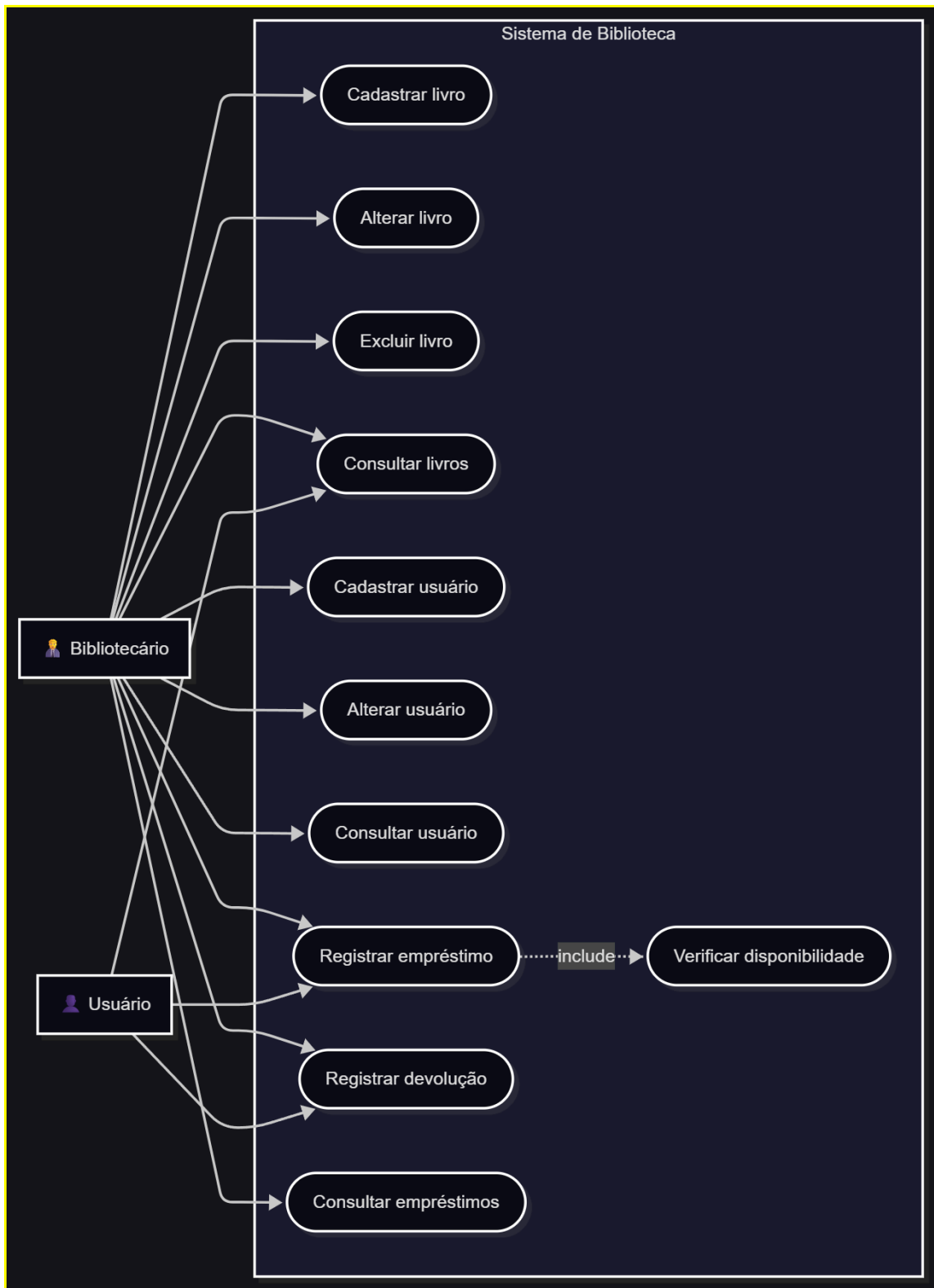
```
Usuario --> UC4  
Usuario --> UC8  
Usuario --> UC9
```

```
UC8 -.->|include| UC11
```

Explicação: `subgraph` cria um agrupamento visual no fluxograma. O texto entre `[""]` define o nome exibido para esse agrupamento.

Em um `flowchart`, `-->` cria uma conexão visual entre dois elementos. Exemplo: o Bibliotecário está relacionado ao caso de uso UC1.

Executar diagrama no VSCode e adicionar o PNG:



2. Diagrama de Classes

Aqui temos as principais entidades do sistema:

- Livro

- Usuario
- Emprestimo
- Bibliotecario

O empréstimo relaciona um usuário a um livro.

classDiagram

```

class Livro {
    +int idLivro
    +String titulo
    +String autor
    +String isbn
    +String editora
    +int anoPublicacao
    +String categoria
    +int quantidade
    +cadastrar()
    +alterar()
    +excluir()
    +consultar()
    +verificarDisponibilidade()
}

class Usuario {
    +int idUsuario
    +String nome
    +String cpf
    +String email
    +String telefone
    +String endereco
    +cadastrar()
    +alterar()
    +consultar()
}

class Emprestimo {
    +int idEmprestimo
    +Date dataEmprestimo
    +Date dataDevolucaoPrevista
    +Date dataDevolucaoReal
    +String status
    +registrarEmprestimo()
    +registrarDevolucao()
    +calcularAtraso()
}

class Bibliotecario {
    +int idBibliotecario
    +String nome
    +String login
    +String senha
    +realizarCadastro()
    +gerenciarEmprestimo()
}

Usuario "1" --> "0..*" Emprestimo : realiza
Livro "1" --> "0..*" Emprestimo : participa
Bibliotecario "1" --> "0..*" Emprestimo : registra

```

Como interpretar

A relação:

Usuario "1" --> "0..*" Emprestimo

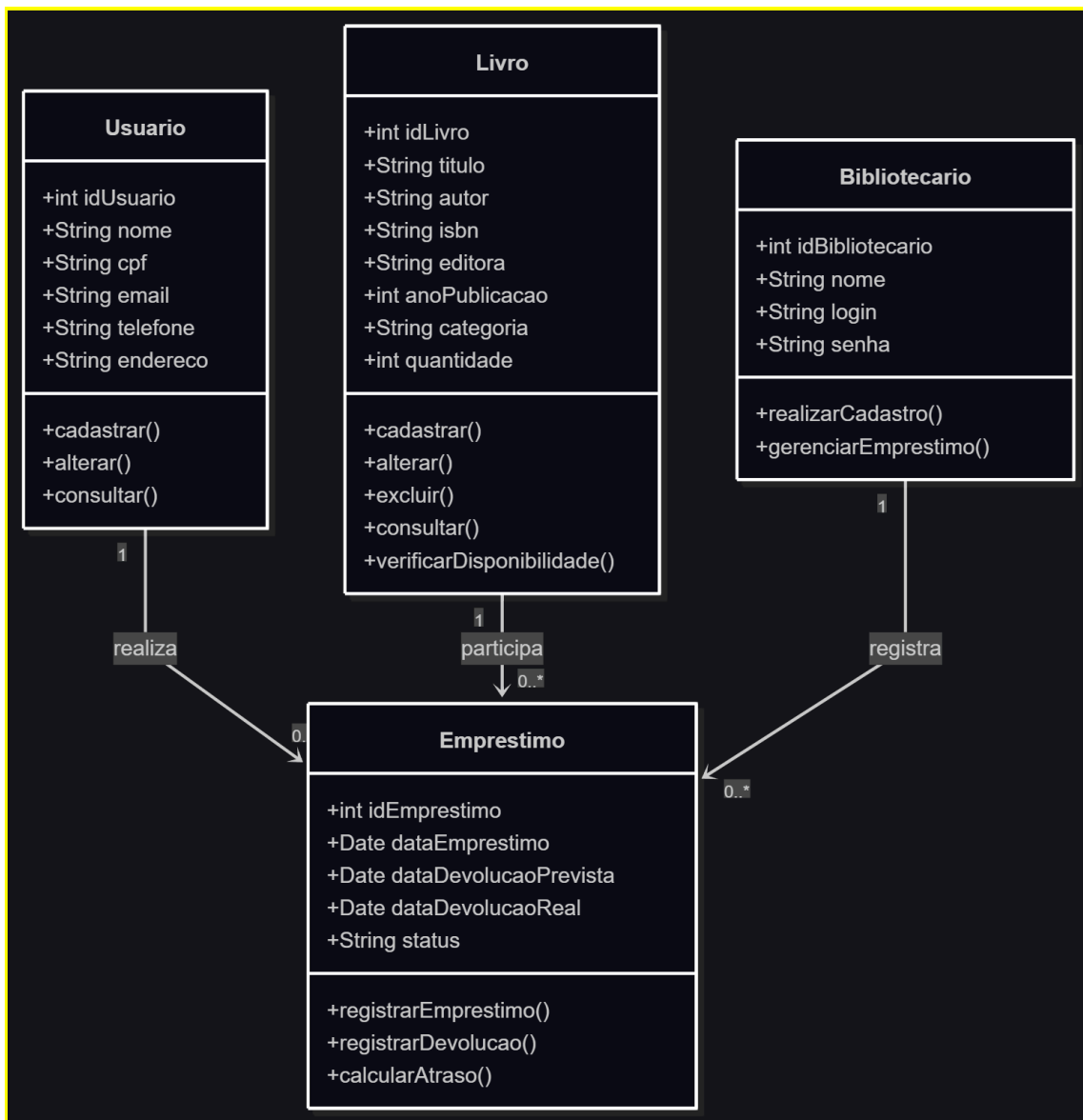
significa que **um usuário pode possuir vários empréstimos**.

Já:

Livro "1" --> "0..*" Emprestimo

indica que um livro pode aparecer em vários registros de empréstimo ao longo do tempo.

Executar diagrama no VSCode e adicionar o PNG:



3. Diagrama de Atividades — Registrar Empréstimo

Este diagrama representa o fluxo principal para realizar um empréstimo.

flowchart TD

```
Inicio([Início])

A["Bibliotecário informa o usuário"]
B["Sistema localiza o usuário"]
C{"Usuário cadastrado?"}

D["Informar o livro"]
E["Sistema verifica o livro"]
F{"Livro cadastrado?"}

G["Verificar disponibilidade"]
H{"Livro disponível?"}

I["Registrar empréstimo"]
J["Atualizar disponibilidade do livro"]
K["Exibir confirmação"]

L["Exibir mensagem: usuário não cadastrado"]
M["Exibir mensagem: livro não cadastrado"]
N["Exibir mensagem: livro indisponível"]

Fim([Fim])

Inicio --> A
A --> B
B --> C

C -- Não --> L
L --> Fim

C -- Sim --> D
D --> E
E --> F

F -- Não --> M
M --> Fim

F -- Sim --> G
G --> H

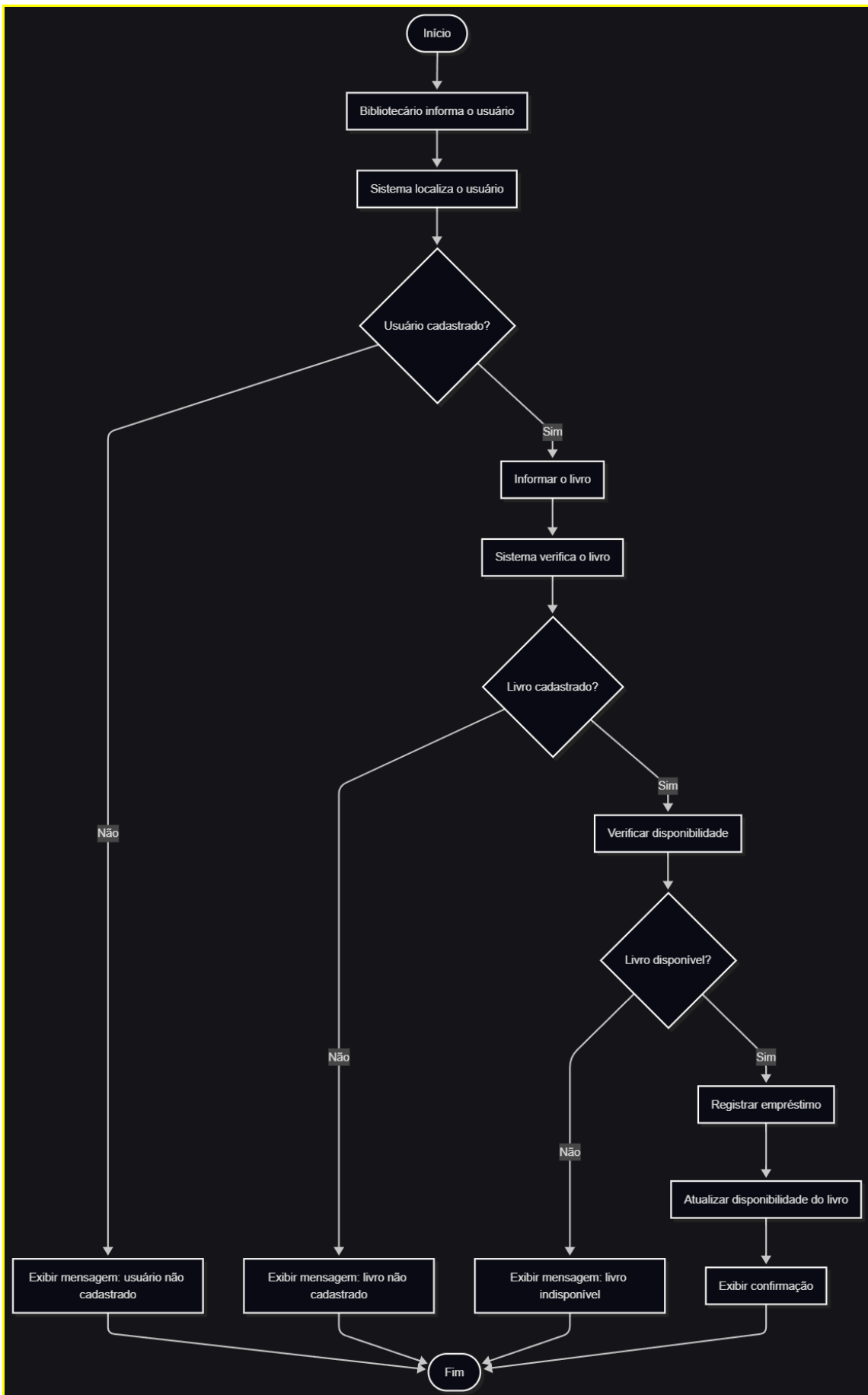
H -- Não --> N
N --> Fim

H -- Sim --> I
I --> J
J --> K
K --> Fim
```

Fluxo resumido

Início → identificar usuário → verificar usuário → identificar livro → verificar livro → verificar disponibilidade → registrar empréstimo → atualizar disponibilidade → finalizar.

Executar diagrama no VSCode e adicionar o PNG:



4. Diagrama de Sequência — Registrar Empréstimo

Este mostra **quem se comunica com quem e em qual ordem** durante o empréstimo.

sequenceDiagram

```
actor Bibliotecario as Bibliotecário
participant Sistema as Sistema de Biblioteca
participant Usuario as Usuário
participant Livro as Livro
participant Empréstimo as Empréstimo

Bibliotecario->>Sistema: Solicitar empréstimo
Sistema->>Usuario: Consultar usuário
Usuario-->>Sistema: Retornar dados do usuário

alt Usuário não cadastrado
    Sistema-->>Bibliotecario: Informar usuário não cadastrado
else Usuário cadastrado

    Bibliotecario->>Sistema: Informar livro
    Sistema->>Livro: Verificar disponibilidade
    Livro-->>Sistema: Retornar disponibilidade

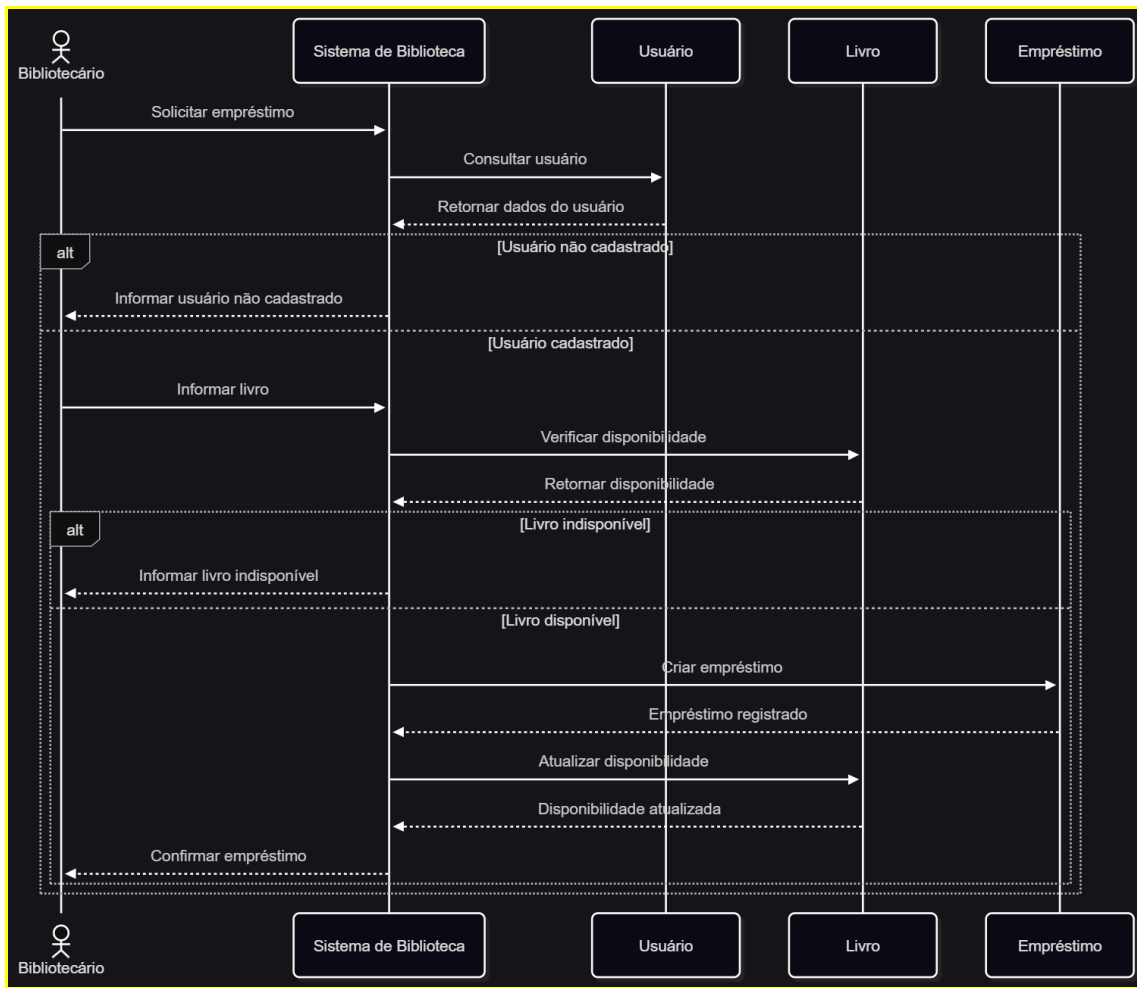
    alt Livro indisponível
        Sistema-->>Bibliotecario: Informar livro indisponível
    else Livro disponível

        Sistema->>Empréstimo: Criar empréstimo
        Empréstimo-->>Sistema: Empréstimo registrado

        Sistema->>Livro: Atualizar disponibilidade
        Livro-->>Sistema: Disponibilidade atualizada

        Sistema-->>Bibliotecario: Confirmar empréstimo
    end
end
```

Executar diagrama no VSCode e adicionar o PNG:



Como os quatro diagramas se relacionam

Diagrama	Pergunta que responde
Caso de Uso	O que o sistema permite fazer?
Classes	Quais são os objetos/entidades do sistema?
Atividades	Qual é o fluxo para realizar uma operação?
Sequência	Como os objetos se comunicam durante a operação?

Questionário – Mermaid e Diagramas UML

1. No código do diagrama de caso de uso, qual é a função de:

```
subgraph Sistema["Sistema de Biblioteca"]
```

- A) Criar uma classe com subgraph chamada Sistema
 - B) Criar um agrupamento visual denominado "Sistema de Biblioteca"(X)
 - C) Criar um banco de dados para o Sistema de Biblioteca
 - D) Criar um dos atores do Sistema de Biblioteca
-

2. Observe o trecho:

Bibliotecario --> UC1

O que a seta --> representa nesse código?

- A) Herança entre duas classes
 - B) Associação entre tabelas SQL
 - C) Relação ou fluxo entre dois elementos(X)
 - D) Criação de um objeto
-

3. No código do caso de uso, temos:

UC1(["Cadastrar livro"])

O que representa UC1?

- A) O nome do ator
 - B) Um identificador interno do elemento(X)
 - C) Uma tabela do banco de dados
 - D) Uma variável do código
-

4. No diagrama de classes encontramos:

```
class Livro {  
    +int idLivro  
    +String titulo  
    +cadastrar()  
}
```

Qual é a finalidade do símbolo +?

- A) Indicar que o atributo ou método é público(X)
 - B) Indicar que o atributo é privado
 - C) Indicar herança pública
 - D) Indicar uma chave estrangeira
-

5. Considere o seguinte código:

Usuario "1" --> "0..*" Empréstimo : realiza

O que significa "0..*"?

- A) Exatamente zero empréstimos
 - B) Exatamente um empréstimo
 - C) De zero a vários empréstimos(X)
 - D) No máximo dois empréstimos
-

6. No diagrama de atividades, temos:

```
C{"Usuário cadastrado?"}
```

```
C -- Não --> L
```

```
C -- Sim --> D
```

Qual é a finalidade da estrutura acima?

A) Criar uma classe para os usuários cadastrados

B) Representar uma decisão com diferentes caminhos(X)

C) Criar um ator chamado usuário

D) Criar uma tabela para cadastro de usuários

7. No diagrama de atividades, qual elemento representa o início do processo?

```
Inicio([Início])
```

A) Inicio

B) [Início]

C) ([Início])(X)

D) Inicio()

8. No diagrama de sequência encontramos:

```
Bibliotecario->>Sistema: Solicitar empréstimo
```

O que essa linha representa?

A) O Bibliotecário herda características do Sistema

B) O Bibliotecário envia uma mensagem para o Sistema(X)

C) O Sistema cria um novo Bibliotecário

D) O Bibliotecário solicita Empréstimo

9. Observe:

```
alt Usuário não cadastrado
```

```
    Sistema-->>Bibliotecario: Informar usuário não cadastrado
```

```
else Usuário cadastrado
```

```
    Bibliotecario->>Sistema: Informar livro
```

```
end
```

Qual é a finalidade de alt e else?

A) Representar alternativas condicionais(X)

B) Criar duas classes

C) Criar dois atores

D) Representar uma relação de herança

10. Qual alternativa apresenta corretamente a associação entre cada tipo de diagrama e sua respectiva sintaxe Mermaid?

A)

- Caso de uso → sequenceDiagram

- Classes → classDiagram
- Sequência → flowchart

B)

- Caso de uso → flowchart
- Classes → classDiagram
- Atividades → flowchart
- Sequência → sequenceDiagram (X)

C)

- Caso de uso → classDiagram
- Classes → flowchart
- Atividades → sequenceDiagram

D)

- Todos os diagramas → classDiagram